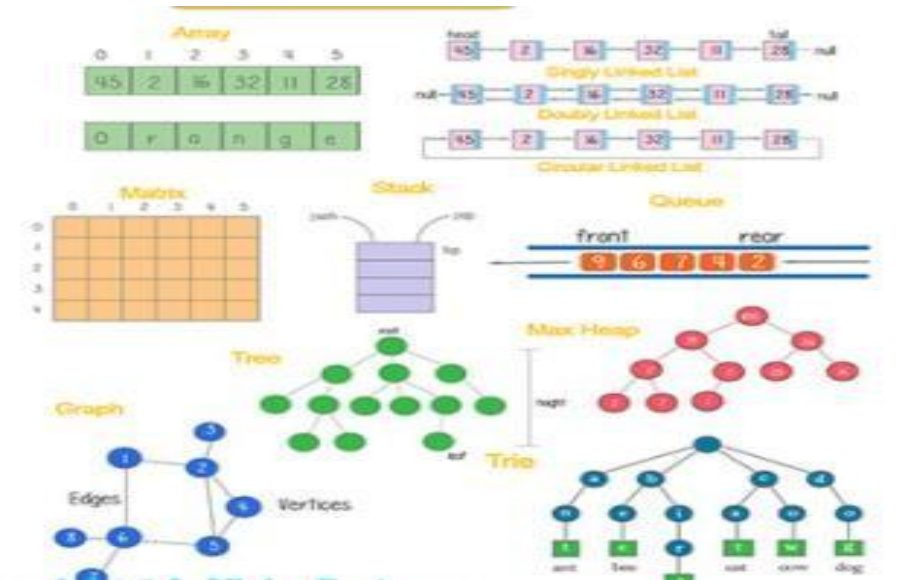
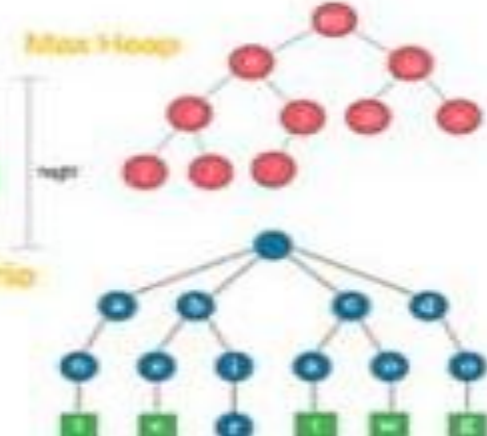
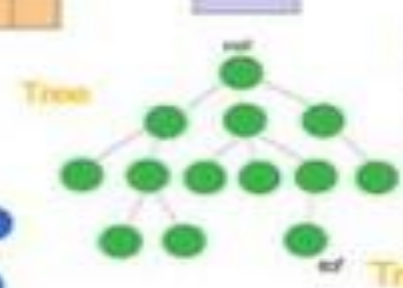
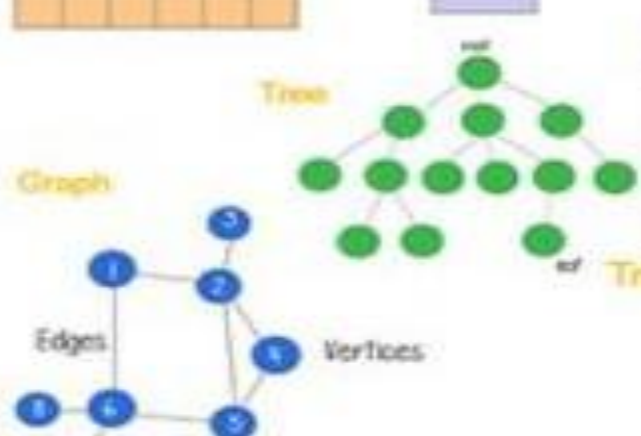
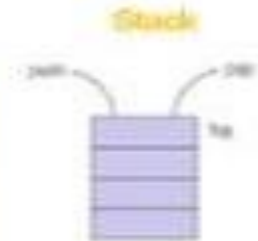
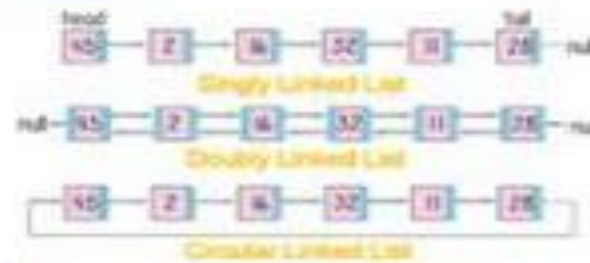


Data Structures

Dr. Elakkiya E
Asst. Professor,
Dept. of CSE, SRM University AP



Data structures



UNIT-V



SEARCHING AND SORTING

Aspects of Computer Programming (Data structures and Algorithms)

- Data organization also commonly called as data structures.
- Choosing the appropriate algorithm to manipulate the data to solve a problem efficiently.

Searching is used to find the location where an element is available.

- 1. Linear or sequential search
- 2. Binary search or half interval search

Sorting allows an efficient arrangement of elements within a given data structure.

- 1. Selection sort
- 2. Bubble sort
- 3. Insertion sort
- 4. Quick sort
- 5. Heap sort
- 6. Radix sort
- 7. Merge sort

Internal sorting techniques (data sorting process that takes place entirely within the main memory).

External sorting technique (some of the data (elements) to be sorted are kept on the secondary storage).

Searching techniques

Linear Search or Sequential Search

```
int main()
{
    int SIZE = 8, key;
    int a1[8]={8, 4, 5, 3, 2, 9, 4, 1};
    printf("Enter the key to search");
    scanf("%d", &key);
    if(linearSearch(a1, SIZE, key) == 1)
        printf("Search Found");
    else
        printf("Search Not Found");
}
```

Search key is compared with each element of the array linearly.

Linear search has complexity of $O(n)$.

```
int linearSearch(int arr[], int size, int key)
{
    int i;
    for(i = 0; i < size; i++)
        if(arr[i] == key)
            return 1;
    return 0;
}
```

Linear Search or Sequential Search

```
int main()
{
    int SIZE = 8;
    int a1[SIZE]={8, 4, 5, 3, 2, 9, 4, 1};
    linearSearch(a1, SIZE, 8);
    linearSearch(a1, SIZE, 4);
    linearSearch(a1, SIZE, 99);
}
```

```
int linearSearch(int a[], int
size, int key)
{
    int flag=0;
    for (int i = 0; i < size; ++i)
    {
        if (a[i] == key)
        {
            flag=1;
            break;
        }
    }
    if(flag==1)
    {
        printf("Key found at %d", i);
        return;
    }
    else
        printf("Key not found");
}
```

Binary Search

- ▶ A **binary search** (or **half-interval search**) is **applicable only to a sorted array**.
- ▶ It compares the **search key** with the **middle element**.
- ▶ If there is a match,
 it returns the element's index.
- ▶ If the search key is less than the middle element,
 repeat searching on the **left half**;
 otherwise, repeat searching on the **right half**.

Binary Search

Array

10	20	35	45	67	78	90	100
----	----	----	----	----	----	----	-----

Key

67

Start Index

End Index

mid

arr[mid]

0

1

2

3

4

5

6

7

0

7

$(0+7) / 2 = 3$

45

10

20

35

45

67

78

90

100

45 is less than 67. So skip first half array. Now start index will be mid + 1. i.e
 $3 + 1 = 4$

4

7

$(4+7) / 2 = 5$

78

67

78

90

100

78 is greater than 67. So skip second half array. Now end index will be mid - 1. i.e
 $5 - 1 = 4$

4

4

$(4+4) / 2 = 4$

67

67

Search Found

Binary Search

Array

10	20	35	45	67	78	90	100
----	----	----	----	----	----	----	-----

Key

50

Start Index

End Index

mid

arr[mid]

0

1

2

3

4

5

6

7

0

7

$(0+7) / 2 = 3$

45

10

20

35

45

67

78

90

100

45 is less than 50. So skip first half array. Now start index will be mid + 1. i.e
 $3 + 1 = 4$

4

7

$(4+7) / 2 = 5$

78

67

78

90

100

78 is greater than 50. So skip second half array. Now end index will be mid - 1. i.e
 $5 - 1 = 4$

4

4

$(4+4) / 2 = 4$

67

67

67 greater than 50. so end 4 - 1 which is 3

4

3

Invalid index range.

Search Not Found

```
int BinarySearch(int arr[],int start,int end,int key)
{
    int mid;
    while(start <= end)
    {
        mid = (start + end) / 2;
        if(arr[mid] == key)
            return 1;

        if(arr[mid] < key)
            start = mid + 1;
        else
            end = mid - 1;
    }
    return 0;
}
```

```
int main()
{
    int SIZE=8, key;
    int a[SIZE] = {10,23,45,70,90,100,111,123};
    printf("Enter the key to search");
    scanf("%d", &key)

    if(BinarySearch(a,0,SIZE - 1, key) == 1)
        printf("Search Found\n");
    else
        printf("Search Not Found\n");

}
```

Sorting techniques

Selection Sort

Method:

- ▶ First smallest element is selected from the unsorted array and placed at the first position.
- ▶ After that second smallest element is selected and placed in the second position.
- ▶ The process continues until the array is entirely sorted.

Unsorted array: {8, 4, 5, 3, 2, 9, 4, 1}

Assumed smallest element Actual smallest element

{ } {8, 4, 5, 3, 2, 9, 4, 1} => { } {1, 4, 5, 3, 2, 9, 4, 8}

Swap elements

{1} {4, 5, 3, 2, 9, 4, 8} => {1} {2, 5, 3, 4, 9, 4, 8}

{1, 2} {5, 3, 4, 9, 4, 8} => {1, 2} {3, 5, 4, 9, 4, 8}

{1, 2, 3} {5, 4, 9, 4, 8} => {1, 2, 3} {4, 5, 9, 4, 8}

{1, 2, 3, 4} {5, 9, 4, 8} => {1, 2, 3, 4} {4, 9, 5, 8}

{1, 2, 3, 4, 4} {9, 5, 8} => {1, 2, 3, 4, 4} {5, 9, 8}

{1, 2, 3, 4, 4, 5} {9, 8} => {1, 2, 3, 4, 4, 5} {8, 9}

{1, 2, 3, 4, 4, 5, 8, 9}

Selection Sort

```
int main()
{
    int SIZE=8;
    int a[SIZE] = {8,4,5,3,2,9,4,1},i;
    selectionSort(a,SIZE);
}
```

Time complexity = $O(n^2)$.

Note:

Stable sorting algorithms preserve the relative order of equal elements, while unstable sorting algorithms don't.

In other words, stable sorting maintains the position of two equals elements relative to one another.

Selection sort is stable algorithm

```
void selectionSort(int a[], int size)
{
    int temp; // for swaping
    for (int i = 0; i < size - 1; ++i)
    {
        int minIndex = i; // assume first element is
                           // the smallest
        for (int j = i + 1; j < size; ++j)
        {
            if (a[j] < a[minIndex])
                minIndex = j;
        }
        if (minIndex != i)
        {
            temp = a[i];
            a[i] = a[minIndex];
            a[minIndex] = temp;
        }
    }
    printf("After selection sort\n");
    for(i = 0; i < size; i++)
        printf("%d ",a[i]);
}
```

Selection Sort

```
int main()
{
    int SIZE=5;
    int arr[SIZE] = {180,165,150,170,145},i;
    selectionSort(arr,SIZE);

    printf("After selection sort\n");
    for(i = 0; i < SIZE; i++)
        printf("%d ",arr[i]);
}
```

```
void swap(int *x, int *y)
{
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

```
void selectionSort(int arr[],int size)
{
    int i,j;

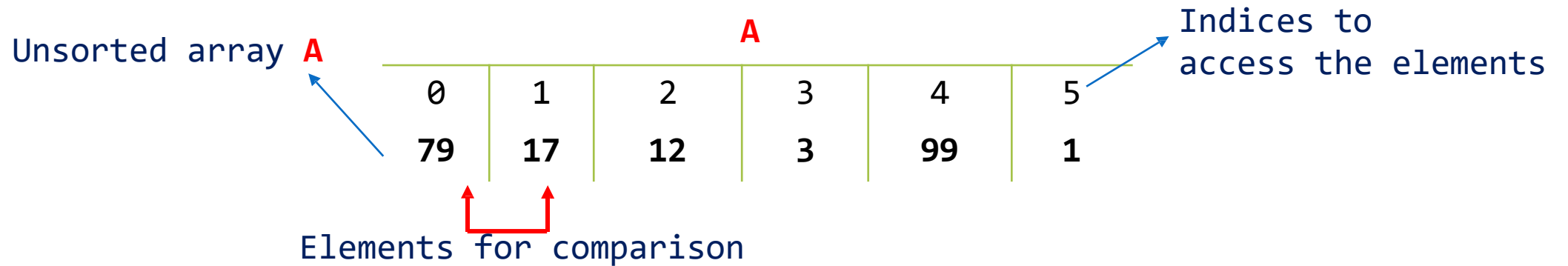
    for(i = 0; i < size-1; i++)
    {
        for(j = i+1; j < size; j++)
        {
            if(arr[i] > arr[j])
                swap(&arr[i],&arr[j]);
        }
    }

    Time complexity =  $O(n^2)$ .
}
```

Bubble Sort

Method:

- ▶ Pass through the list,
- ▶ Compare two adjacent elements
swap them if they are in the wrong order.
- ▶ Repeat the pass until no swaps are needed.



Is $A[0] > A[1]$?
Yes
So swap

0	1	2	3	4	5
17	79	12	3	99	1

Pass 1
Comparison 1
is over

Bubble Sort

Is $A[1] > A[2]$?
Yes
So swap these two

0	1	2	3	4	5
17	12	79	3	99	1



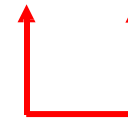
Is $A[2] > A[3]$?
Yes
So swap these two

0	1	2	3	4	5
17	12	3	79	99	1



Is $A[3] > A[4]$?
No
Don't swap

0	1	2	3	4	5
17	12	3	79	99	1



Is $A[4] > A[5]$?
Yes
So swap these two

0	1	2	3	4	5
17	12	3	79	1	99



Pass 1
Comparison 2
is over

Pass 1
Comparison 3
is over

Pass 1
Comparison 4
is over

Pass 1
Comparison 5
is over

Bubble Sort

Is $A[0] > A[1]$?
Yes
So swap these two

Is $A[1] > A[2]$?
Yes
So swap these two

Is $A[2] > A[3]$?
No
Don't swap

Is $A[3] > A[4]$?
Yes
So swap these two

Is $A[4] > A[5]$?
No
Don't swap

0	1	2	3	4	5
12	17	3	79	1	99



0	1	2	3	4	5
12	3	17	79	1	99



0	1	2	3	4	5
12	3	17	79	1	99



0	1	2	3	4	5
12	3	17	1	79	99



0	1	2	3	4	5
12	3	17	1	79	99



Pass 2
Comparison 1
is over

Pass 2
Comparison 2
is over

Pass 2
Comparison 3
is over

Pass 2
Comparison 4
is over

Pass 2
Comparison 5
is over

Bubble Sort

Is $A[0] > A[1]$?
Yes
So swap these two

Is $A[1] > A[2]$?
No
Don't swap

Is $A[2] > A[3]$?
Yes
So swap these two

Is $A[3] > A[4]$?
No
Don't swap

Is $A[4] > A[5]$?
No
Don't swap

0	1	2	3	4	5
3	12	17	1	79	99



0	1	2	3	4	5
3	12	17	1	79	99



0	1	2	3	4	5
3	12	1	17	79	99



0	1	2	3	4	5
3	12	1	17	79	99



0	1	2	3	4	5
3	12	1	17	79	99



Pass 3
Comparison 1
is over

Pass 3
Comparison 2
is over

Pass 3
Comparison 3
is over

Pass 3
Comparison 4
is over

Pass 3
Comparison 5
is over

Bubble Sort

0	1	2	3	4	5
3	12	1	17	79	99



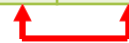
Pass 4
Comparison 1
is over

0	1	2	3	4	5
3	1	12	17	79	99



Pass 4
Comparison 2
is over

0	1	2	3	4	5
3	1	12	17	79	99



Pass 4
Comparison 3
is over

0	1	2	3	4	5
3	1	12	17	79	99



Pass 4
Comparison 4
is over

0	1	2	3	4	5
3	12	1	17	79	99



Pass 4
Comparison 5
is over

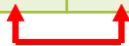
Bubble Sort

0	1	2	3	4	5
1	3	12	17	79	99



Pass 5
Comparison 1
is over

0	1	2	3	4	5
1	3	12	17	79	99



Pass 5
Comparison 2
is over

0	1	2	3	4	5
1	3	12	17	79	99



Pass 5
Comparison 3
is over

0	1	2	3	4	5
1	3	12	17	79	99



Pass 5
Comparison 4
is over

0	1	2	3	4	5
1	3	12	17	79	99



Pass 5
Comparison 5
is over

Bubble Sort

Method:

- Pass through the list,
- Compare two adjacent items
swap them if they are in the wrong order.
- Repeat the pass until no swaps are needed.

Unsorted array: {8,4,5,3,2,9,4,1}

PASS 1 ...

{8,4,5,3,2,9,4,1} => {4,8,5,3,2,9,4,1}
 {4,8,5,3,2,9,4,1} => {4,5,8,3,2,9,4,1}
 {4,5,8,3,2,9,4,1} => {4,5,3,8,2,9,4,1}
 {4,5,3,8,2,9,4,1} => {4,5,3,2,8,9,4,1}
 {4,5,3,2,8,9,4,1} => {4,5,3,2,8,4,9,1}
 {4,5,3,2,8,4,9,1} => {4,5,3,2,8,4,1,9}

PASS 2 ...

{4,5,3,2,8,4,1,9} => {4,3,5,2,8,4,1,9}
 {4,3,5,2,8,4,1,9} => {4,3,2,5,8,4,1,9}
 {4,3,2,5,8,4,1,9} => {4,3,2,5,4,8,1,9}
 {4,3,2,5,4,8,1,9} => {4,3,2,5,4,1,8,9}

PASS 3 ...

{4,3,2,5,4,1,8,9} => {3,4,2,5,4,1,8,9}
 {3,4,2,5,4,1,8,9} => {3,2,4,5,4,1,8,9}
 {3,2,4,5,4,1,8,9} => {3,2,4,4,5,1,8,9}
 {3,2,4,4,5,1,8,9} => {3,2,4,4,1,5,8,9}

PASS 4 ...

{3,2,4,4,1,5,8,9} => {2,3,4,4,1,5,8,9}
 {2,3,4,4,1,5,8,9} => {2,3,4,1,4,5,8,9}

PASS 5 ...

{2,3,4,1,4,5,8,9} => {2,3,1,4,4,5,8,9}

PASS 6 ...

{2,3,1,4,4,5,8,9} => {2,1,3,4,4,5,8,9}

PASS 7 ...

{2,1,3,4,4,5,8,9} => {1,2,3,4,4,5,8,9}

Sorted array: {1,2,3,4,4,5,8,9}

Bubble Sort

```
void bubbleSort(int arr[])
{
    int i,j;
    for{ j=1; j <= n-1; i++)// Pass number
    {
        for{ i=1; i <= n-1-j; i++)// Comparison
        {
            if(A[i-1] > A[i])
            {
                temp = A[i-1];
                A[i-1] = A[i];
                A[i] = temp;
            }
        }
    }
}
```

Number of passes in bubble sort with
“n” elements is (n-1)

Time complexity = $O(n^2)$.

Bubble sort is stable algorithm

Insertion Sort

■ While some elements are unsorted ...

- Using linear search, find the location in the sorted portion where the 1st element of the unsorted portion should be inserted.
- Move all the elements after the insertion location up one position to make space for the new element.

Initial array: A

Pivot

Element to be processed

29	10	14	37	13
----	----	----	----	----

Is temp < A[i-1]
If yes, then move A[i-1] to A[i]
And insert temp at A[i-1]
i=i+1

29	10	14	37	13
----	----	----	----	----

29	29	14	37	13
----	----	----	----	----

10	29	14	37	13
----	----	----	----	----

10	29	29	37	13
----	----	----	----	----

10	14	29	37	13
----	----	----	----	----

10	14	29	37	13
----	----	----	----	----

10	14	14	29	37
----	----	----	----	----

Sorted array:

10	13	14	29	37
----	----	----	----	----

i=1;

Copy 10 to a temp variable

Shift 29

Insert 10; copy 14

Shift 29

Insert 14; copy 37, insert 37 on top of itself

Copy 13

Shift 37, 29, 14

Insert 13

Insertion Sort

```
void main()
{
    int n, arr[1000], i, d, temp;
    printf("Enter number of elements\n");
    scanf("%d", &n);
    printf("Enter %d integers\n", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    for (i = 1; i <= n - 1; i++) {
        d = i;
        while (d > 0 && arr[d-1] > arr[d]) {
            temp = arr[d];
            arr[d] = arr[d-1];
            arr[d-1] = temp;
            d--;
        }
    }
    printf("Sorted array in ascending order:\n");
    for (i = 0; i <= n - 1; i++) {
        printf("%d\n", arr[i]);
    }
}
```

Data Structures--> Unit-V:

Initial array:

29	10	14	37	13
----	----	----	----	----

29	29	14	37	13
----	----	----	----	----

10	29	14	37	13
----	----	----	----	----

10	29	29	37	13
----	----	----	----	----

10	14	29	37	13
----	----	----	----	----

10	14	29	37	13
----	----	----	----	----

10	14	14	29	37
----	----	----	----	----

Sorted array:

10	13	14	29	37
----	----	----	----	----

Time complexity is $O(n^2)$

Insertion sort is stable algorithm

Quick Sort

Quicksort is a **divide and conquer algorithm**.

It divides the large array into smaller sub-arrays.

And then quicksort recursively sort the sub-arrays.

Pivot

1. Picks an element called the **"pivot"**.

There are many ways to choose the **pivot element**.

i) The first element or second or middle or **end element** in the array

ii) We can also pick the element randomly.

Partition

2. Rearrange the array elements such that the all values lesser than the pivot should come before the pivot and all the values greater than the pivot should come after it.

At the end of the partition, the pivot element will be placed at its sorted position.

Recursive

3. Do the above process recursively to all the sub-arrays and sort the elements.

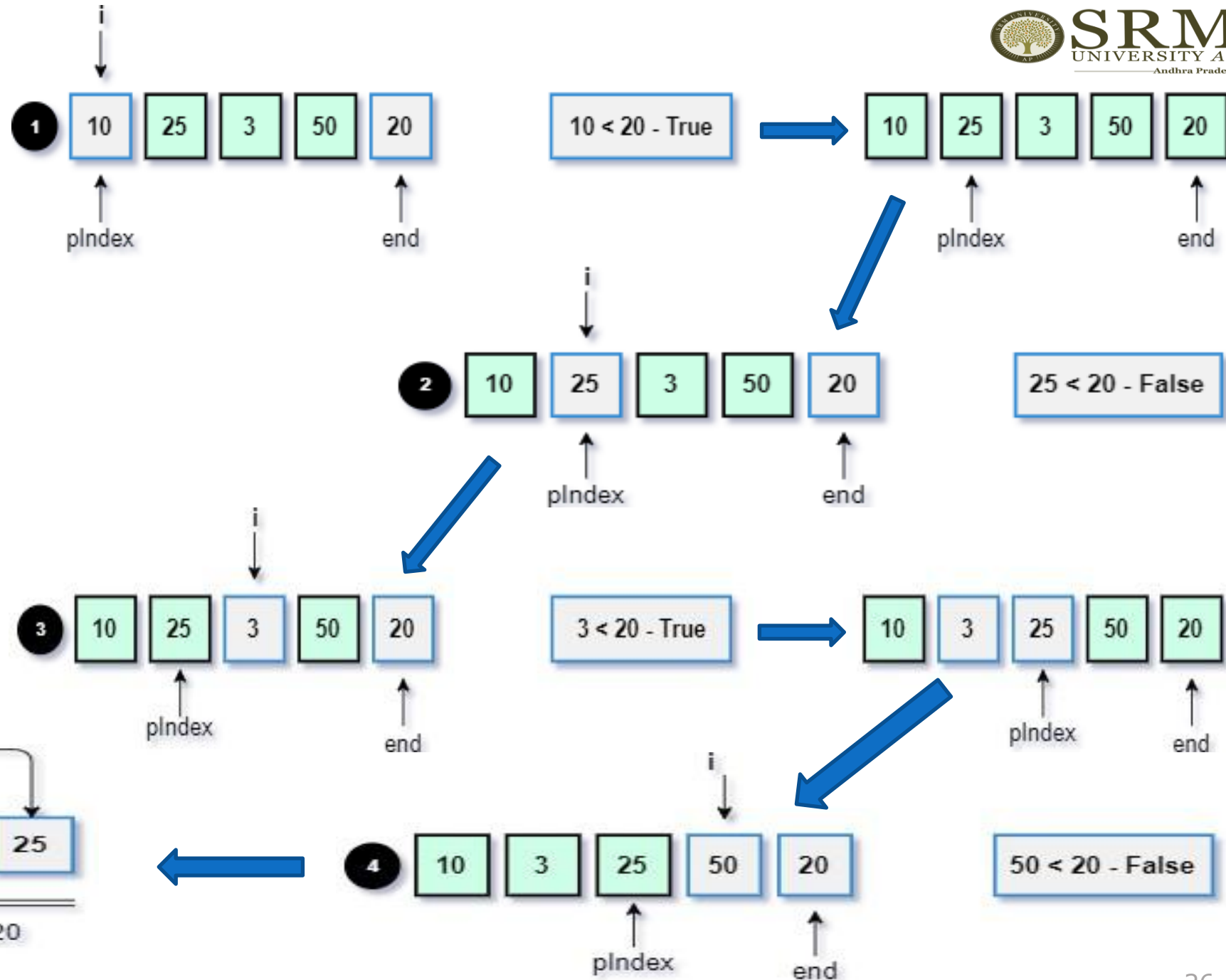
Base Case

If the array has zero or one element, there is no need to call the partition method.

Quick Sort

Example

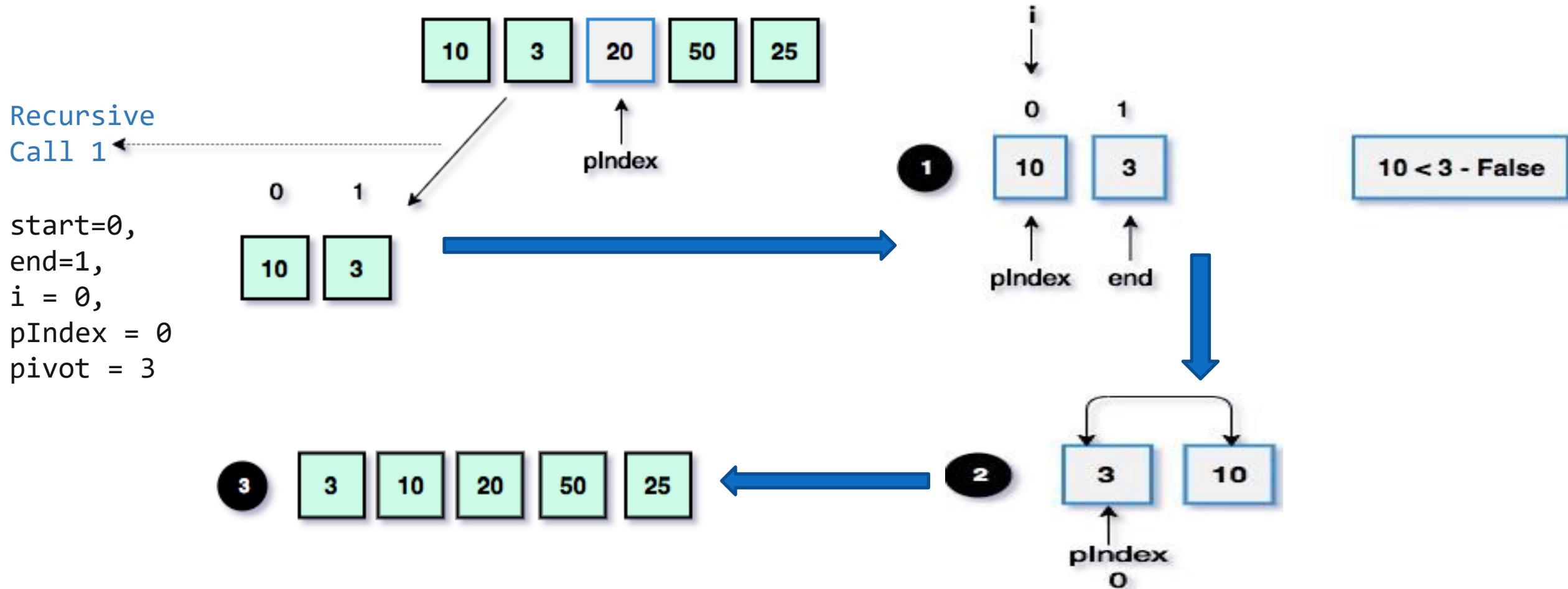
`arr[5] = {10, 25, 3, 50, 20}`
`start = 0, end = 4,`
`pindex = 0, pivot=arr[4]=20`



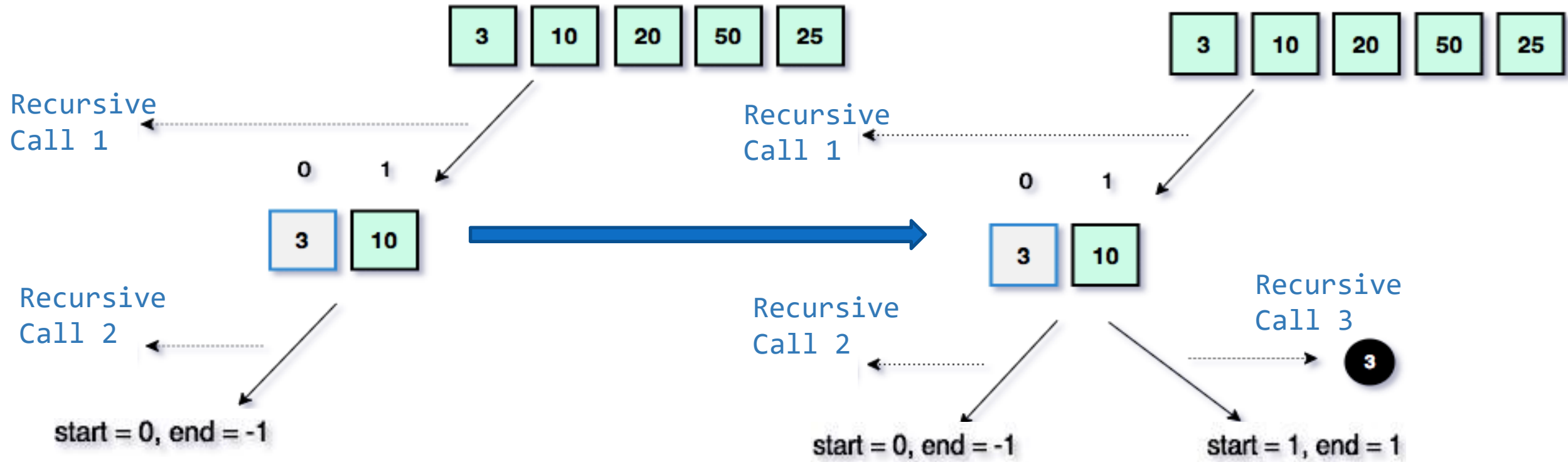
Quick Sort

Recursive calls:

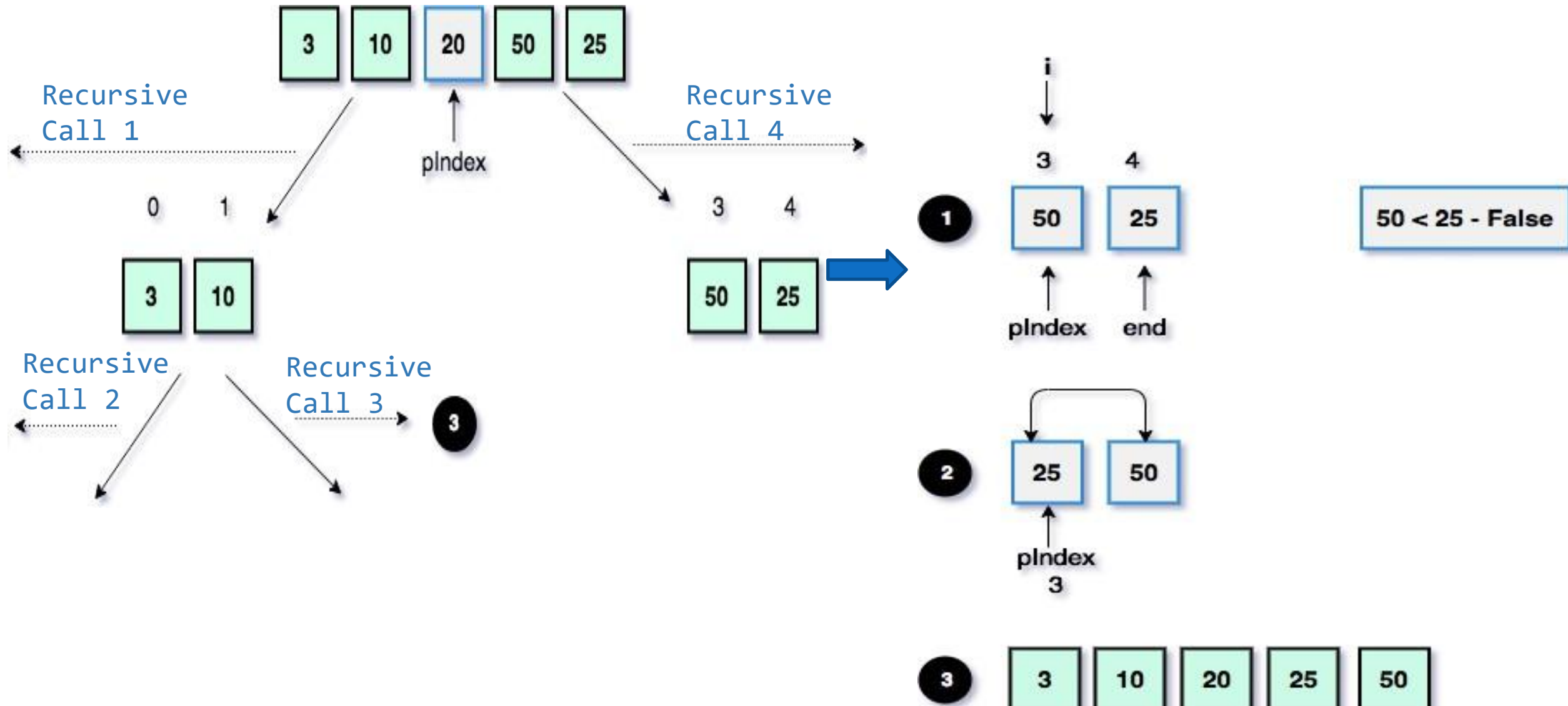
After splitting the array into 2 partitions, quicksort algorithm called recursively on each sub array



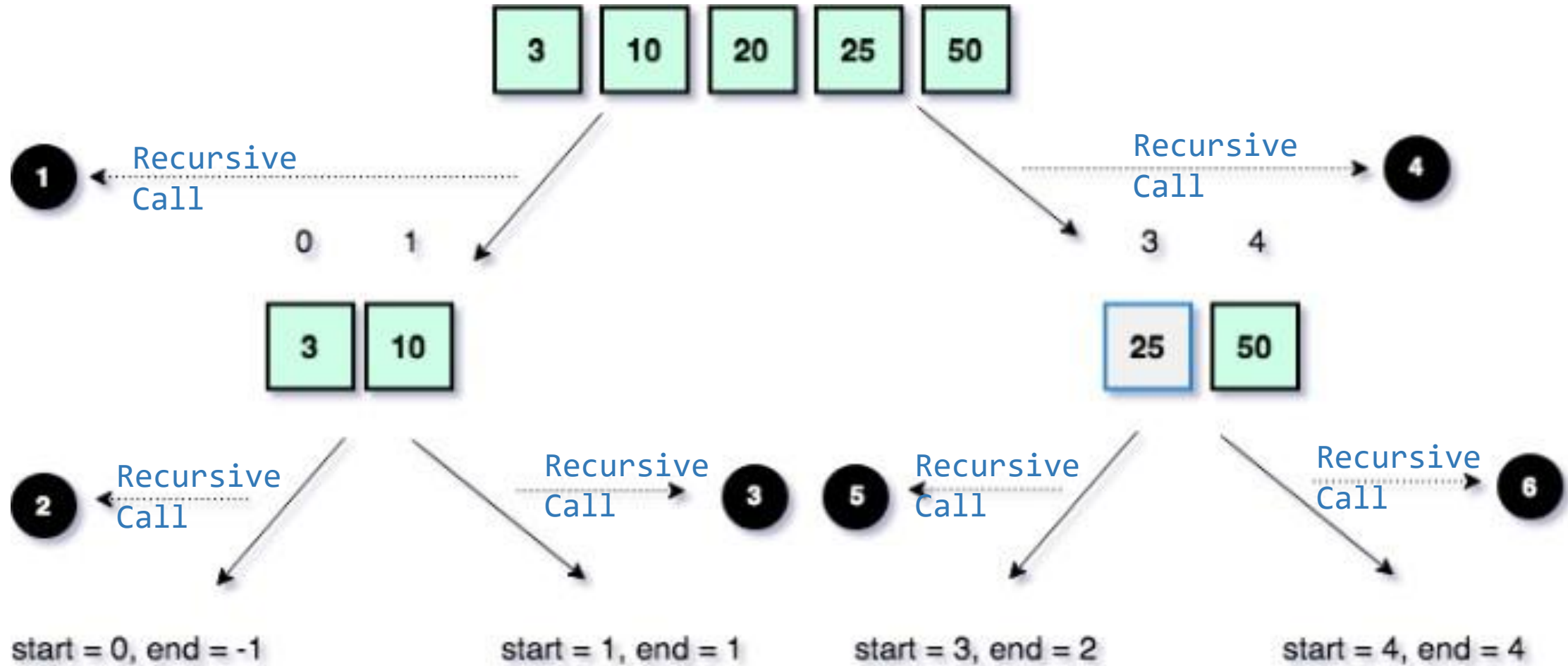
Quick Sort



Quick Sort



Quick Sort



Sorted array: {3, 10, 20, 25, 50}

Quick Sort

```
void quickSort(int arr[], int start, int end){
    if(start < end)
    {
        int pIndex = partition(arr, start, end);
        quickSort(arr, start, pIndex-1);
        quickSort(arr, pIndex+1, end);
    }
}

int partition(int arr[], int start, int end){
    int pIndex = start;
    int pivot = arr[end];
    int i;
    for(i = start; i < end; i++){
        if(arr[i] < pivot)
        {
            swap(&arr[i], &arr[pIndex]);
            pIndex++;
        }
    }
    swap(&arr[end], &arr[pIndex]);
    return pIndex;
}
```

```
int main(){
    int n,i ;
    printf("Enter Array Size\n");
    scanf("%d",&n);
    int arr[n];
    printf("Enter Array Elements\n");
    for(i=0;i<n;i++)
        scanf("%d",&arr[i]);
    quickSort(arr,0,n-1);
    printf("After the QuickSort\n");
    for(i=0;i<n;i++)
        printf("%d ",arr[i]);
    printf("\n");
    return 0;
}
```

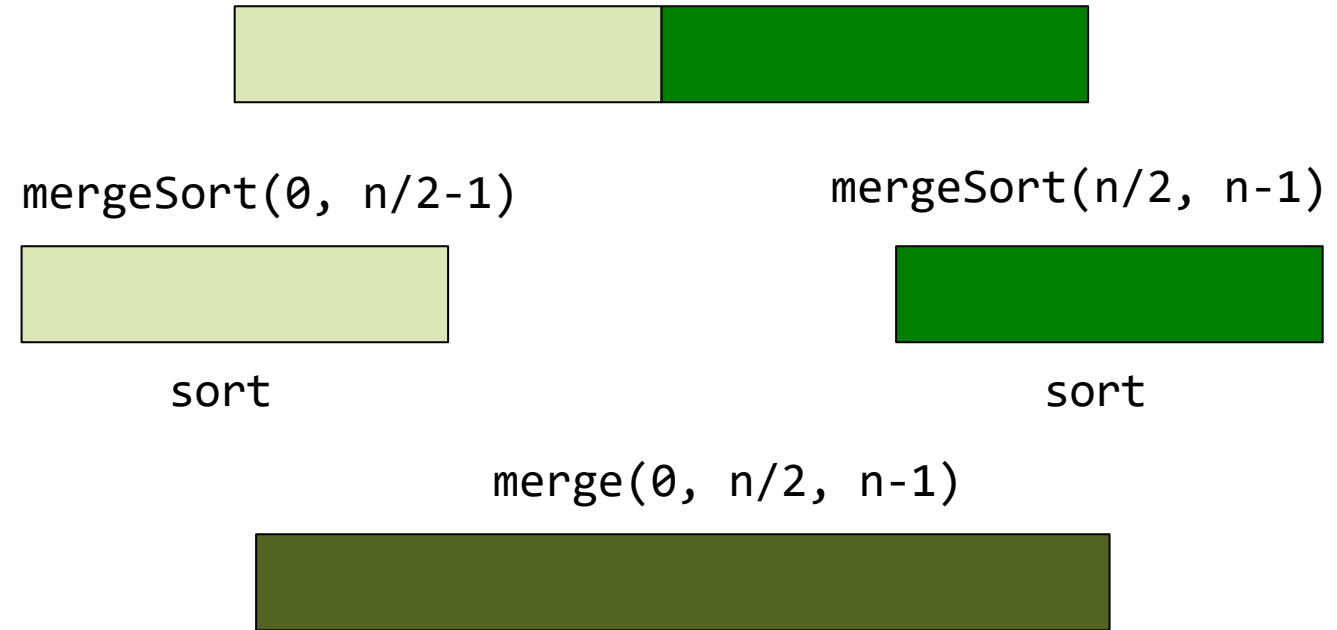
```
void swap(int *x, int *y)
{
    int t = *x;
    *x = *y;
    *y = t;
}
```

Merge Sort

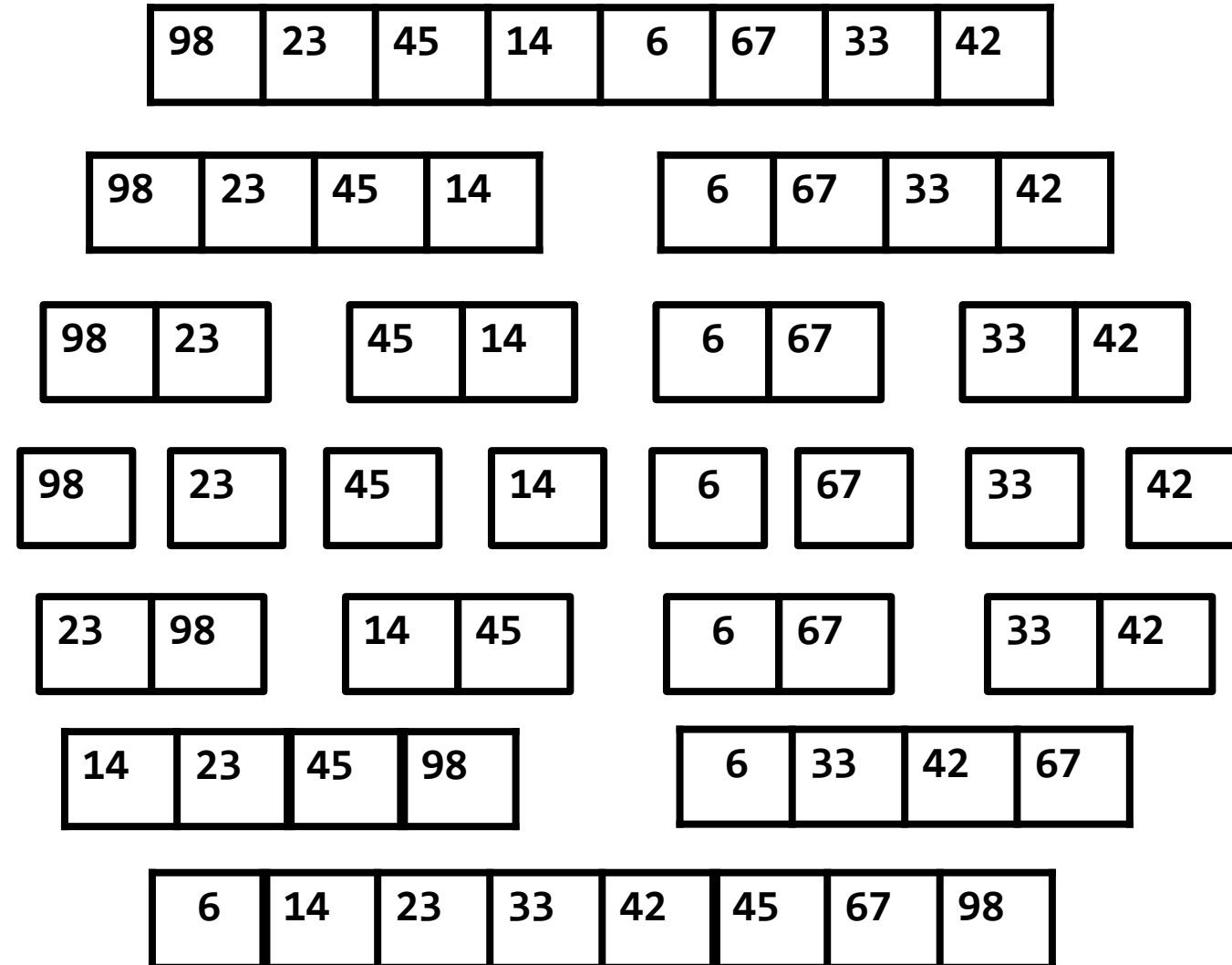
Method :

A "divide and conquer" algorithm

- Divides the array into two roughly equal parts
- Recursively divide each part in half, continuing until a part contains only one element
- Recursively sort the two halves, and merge the two parts into one sorted array.
- Continue to merge parts as the recursion unfolds

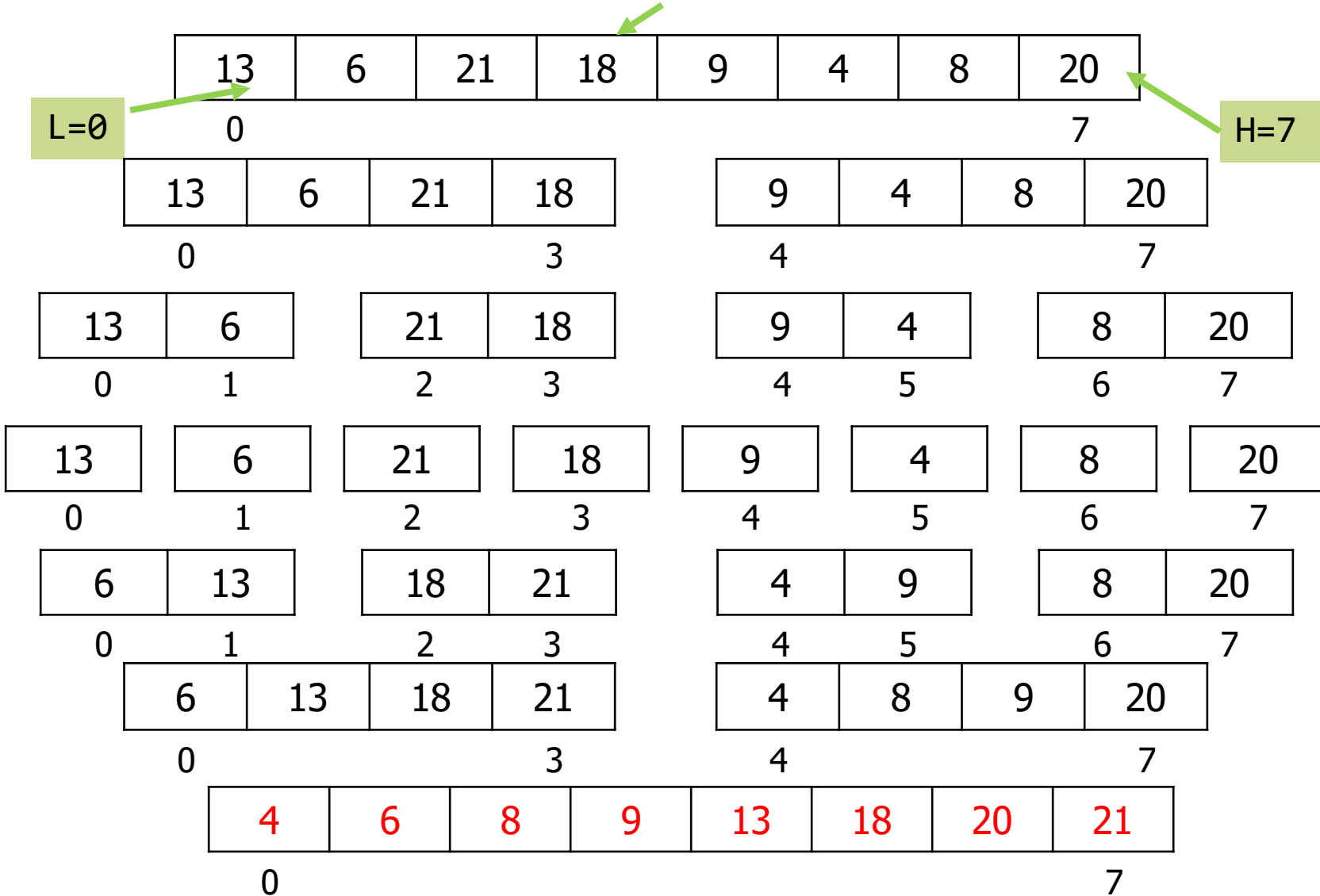


Merge Sort



Merge Sort

$$M = \text{floor}((L+H)/2)$$



Algorithm:

```
MergSort(A, L, H)
{
    if(L<H)
    {
        mid = floor((L+H)/2)
        MergSort(A, L, mid)
        MergSort(A, mid+1, H)
        MergingArrays(A1, A2, L,H)
    }
}
```

- *Merge operation:*
 - Given two sorted arrays, *merge* operation produces a sorted array with all the elements of the two arrays

A	6	13	18	21	B	4	8	9	20
C	4	6	8	9	13	18	20	21	

Running time of *merge*: $O(n)$, where n is the number of elements in the merged array.

Algorithm for merging two given sorted arrays:

MergingTwoArrays()

```
{
While( i < len1  && j < len2)
{
    if(A[i] < B[j])
    {
        MergedArray[k++] = A[i++];
    }
    else
    {
        MergedArray[k++] = B[j++];
    }
}
// Copy rest of the elements as it is...
While( i < len1)
    MergedArray[k++] = A[i++];
While( j < len2)
    MergedArray[k++] = B[j++];
}
```

Merge Sort

```
int main()
{
    int a[30],n,i;
    printf("Enter no of elements:");
    scanf("%d",&n);
    printf("Enter array elements:");

    for(i=0;i<n;i++)
        scanf("%d",&a[i]);

    mergesort(a,0,n-1);

    printf("\nSorted array is :");
    for(i=0;i<n;i++)
        printf("%d ",a[i]);

    return 0;
}

void mergesort(int a[],int i,int j)
{
    int mid;

    if(i<j)
    {
        mid=(i+j)/2;
        mergesort(a,i,mid); //left recursion
        mergesort(a,mid+1,j); //right recursion
        merge(a,i,mid,mid+1,j); //merging of two
    }
}
```

```
void merge(int a[],int i1,int j1,int i2,int j2)
{
    int temp[50]; //array used for merging
    int i,j,k;
    i=i1; //beginning of the first list
    j=i2; //beginning of the second list
    k=0;

    while(i<=j1 && j<=j2) //while elements in both lists are not exhausted
    {
        if(a[i]<a[j])
            temp[k++]=a[i++];
        else
            temp[k++]=a[j++];
    }

    while(i<=j1) //copy remaining elements of first list
        temp[k++]=a[i++];

    while(j<=j2) //copy remaining elements of second list
        temp[k++]=a[j++];

    //Transfer elements from temp[] back to a[]
    for(i=i1,j=0;i<=j2;i++,j++)
        a[i]=temp[j];
}
```

Merge Sort

Algorithm:

MergSort(A, L, H)

```
{  
    if(L<H)  
    {  
        mid = floor((L+H)/2))  
        MergSort(L, mid)      → T(n/2)  
        MergSort(mid+1, H)    → T(n/2)  
        Merge(L,H)            → T(n)  
    }  
}
```

- Divide the unsorted collection into two ($O(\log n)$)
-
- Until the sub-arrays only contain one element ($O(n)$)
- Then merge the sub-problem solutions together
- Total runtime $O(n \log n)$.

Note: An in-place algorithm is an algorithm that does not need an extra space and produces an output in the same memory that contains the data by transforming the input 'in-place'. However, a small constant extra space used for variables is allowed.

Merge Sort is not in-place algorithm, selection, bubble, insertion, quick, heap sort algorithms are in-place algorithms

In Radix sort, digit by digit sorting is performed starting from the least significant digit to the most significant digit.



Radix sort works similar to the sorting of names, according to the alphabetical order.

0-9 (10) buckets required for sorting the numbers

A-Z (26) buckets required for sorting the names

Radix Sort

Unsorted array:

181	289	390	121	145	736	514	212
-----	-----	-----	-----	-----	-----	-----	-----

0-9 buckets required

Largest element: 736, three passes require

First pass:

181		
289		
390		
121		
145		
736		
514		
212		

390	0
181	1
121	1
212	2
	3
514	4
145	5
736	6
	7
	8
289	9

Buckets

After the first pass:

390	181	121	212	514	145	736	289
-----	-----	-----	-----	-----	-----	-----	-----

Second pass:

390		
181		
121		
212		
514		
145		
736		
289		

	0
212	1
514	1
121	2
736	3
145	4
	5
	6
	7
181	8
289	8
390	9

After the second pass:

212	514	121	736	145	181	289	390
-----	-----	-----	-----	-----	-----	-----	-----

Third pass:

212		
514		
121		
736		
145		
181		
289		
390		

	0
121	1
145	1
181	2
212	2
289	2
390	3
	4
514	5
	6
736	7
	8
	9

After the third pass:

121	145	181	212	289	390	514	736
-----	-----	-----	-----	-----	-----	-----	-----

Sorted array

Time complexities of different algorithms

Algorithm	Runtime
Linear Search	$O(n)$
Binary Search	$O(\log n)$
Bubble Sort	$O(n^2)$
Insertion Sort	$O(n^2)$
Selection Sort	$O(n^2)$
Merge Sort	$O(n \log n)$
Quick Sort	$O(n^2)$
Heapsort	$O(n \log n)$
Radix sort	$O(n * k)$

Why hashing ?

Linear search has a running time proportional to $O(n)$

Binary search takes time proportional to $O(\log n)$

Is there a way to search an array in constant time $O(1)$, irrespective of its size?

Key	Array of Employees' Records
Key 0 → [0]	Employee record with Emp_ID 0
Key 1 → [1]	Employee record with Emp_ID 1
Key 2 → [2]	Employee record with Emp_ID 2
.....	
.....	
Key 98 → [98]	Employee record with Emp_ID 98
Key 99 → [99]	Employee record with Emp_ID 99

For single digit Emp_ID, we need an array of size 100, of which all 100 elements will be used.

Key	Array of Employees' Records
Key 00000 → [0]	Employee record with Emp_ID 00000
.....	
Key n → [n]	Employee record with Emp_ID n
.....	
Key 99998 → [99998]	Employee record with Emp_ID 99998
Key 99999 → [99999]	Employee record with Emp_ID 99999

For a five-digit Emp_ID, we need an array of size 100,000, of which only 100 elements will be used.

Whether we use a two-digit primary key (Emp_ID) or a five-digit key, there are just 100 employees in the company. Thus, we will be using only 100 locations in the array.

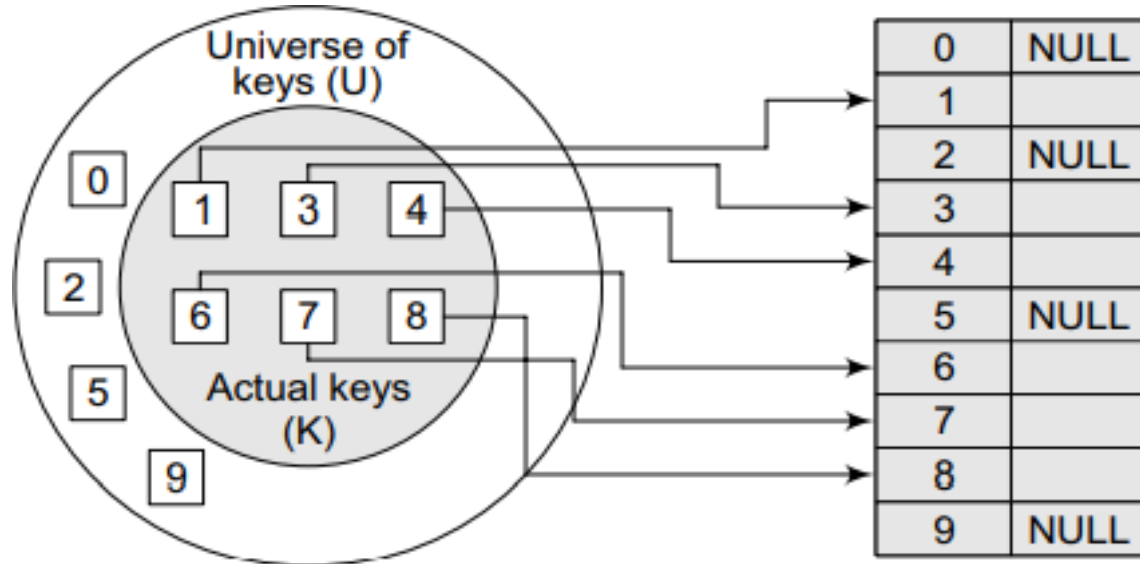
Solution:

- ▶ Elements are not stored according to the **value** of the **key**.
- ▶ We need a way to convert a **five-digit key** number to a **two-digit array index**.
- ▶ We need a **function** which will do the transformation.
- ▶ The term **hash table** is used for an array and the function that will carry out the transformation will be called a **hash function**.

Hash table is a data structure in which keys are mapped to array positions by a hash function.

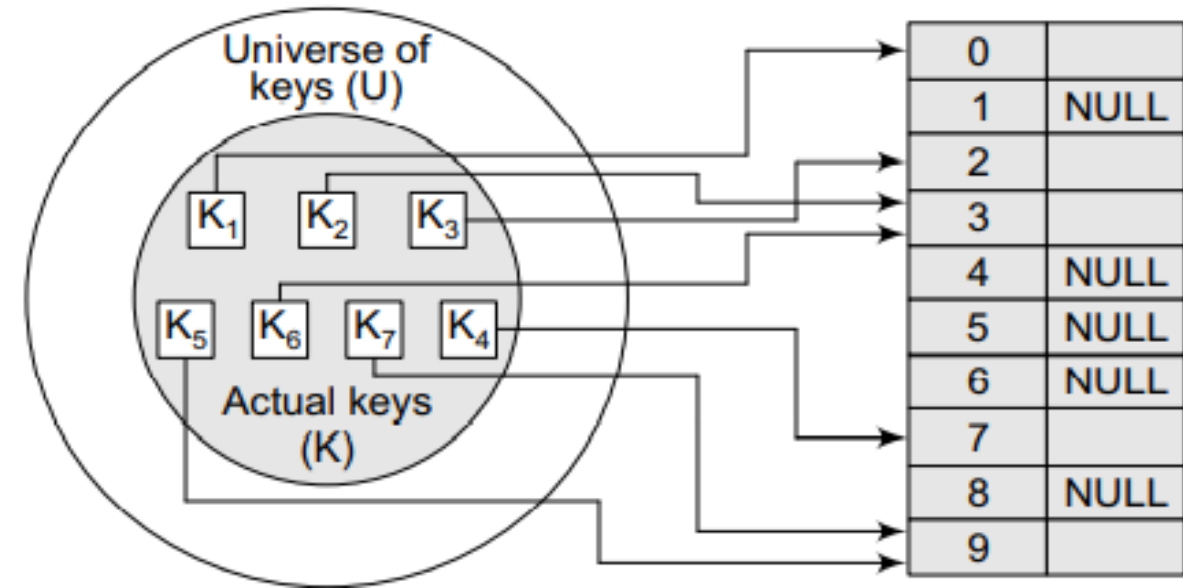
A **hash function** is a mathematical formula which, when applied to a key, produces an integer which can be used as an index for the key in the hash table.

Hashing



Direct relationship between key and index in the array

- Universe of keys is small and when most of the keys are actually used from the whole set of keys.



Indirect Relationship between keys and hash table index

- In a hash table, an element with key k is stored at index $h(k)$ and not k .
- A hash function h is used to calculate the index at which the element with key k will be stored.
- The process of mapping the keys to appropriate locations (or indices) in a hash table is called hashing.

Different hashing functions

1. Division Method:

It is the most simple method of hashing an integer x .

This method divides x by M and then uses the remainder obtained.

The hash function can be given as $h(x) = x \bmod M$

Extra care should be taken to select a suitable value for M .

For example, suppose M is an even number then $h(x)$ is even if x is even and $h(x)$ is odd if x is odd

The division method will not spread the hashed values uniformly.

It is **best to choose M to be a prime number** because making M a prime number increases the likelihood that the keys are mapped with a uniformity in the output range of values.

Example: Hash values of keys 1234 and 5462.

$M = 97$,

$h(1234) = 1234 \% 97 = 70$

$h(5642) = 5642 \% 97 = 16$

Hashing functions

2. Multiplication Method:

Step 1: Choose a constant A such that $0 < A < 1$

Step 2: Multiply the key k by A.

Step 3: Extract the fractional part of kA.

Step 4: Multiply the result of Step 3 by the size of hash table (m).

The hash function can be given as:

$$h(k) = \text{floor}[m * (kA \bmod 1)]$$

Example:

Given a hash table of size 1000, map the key 12345 to an appropriate location in the hash table.

Solution:

Let $A = 0.618033$, $m = 1000$, and $k = 12345$

$$h(12345) = 1000 (12345 * 0.618033 \bmod 1)$$

$$h(12345) = 1000 (7629.617385 \bmod 1)$$

$$h(12345) = 1000 (0.617385)$$

$$h(12345) = 617.385$$

$$h(12345) = 617$$

3. Mid-Square Method:

Step 1: Square the value of the key. That is, find k^2 .

Step 2: Extract the middle r digits of the result obtained in Step 1

Example:

Calculate the hash value for keys 1234 and 5642 using the mid-square method. The hash table has 100 memory locations.

Solution:

Locations whose indices vary from 0 to 99.

Only two digits are needed to map the key to a location in the hash table, so $r = 2$.

When $k = 1234$, $k^2 = 152756$, $h(1234) = 27$

When $k = 5642$, $k^2 = 3183164$, $h(5642) = 21$

Note that the 3rd and 4th digits starting from the right are chosen

Hashing functions

4. Folding Method: